GUIA VISUAL 035

DPKG E OS ARQUIVOS .DEB

O `dpkg -i` falhou, o pacote ficou esquisito — e agora o APT inteiro se recusa a rodar.

035
GUIAS LINUX

UM `.DEB` É UM ARQUIVO `AR` COM TRÊS MEMBROS

`nmap_7.95+dfsg-3_amd64.deb` — e nada além destes três

`debian-binary`
Uma linha: `2.0`, a versão do **formato** do pacote.

`control.tar.xz`
`control` — nome, versão, arquitetura, `Depends`, descrição.
`md5sums` — hash de cada arquivo · `conffiles` — o que é configuração.
`preinst` · `postinst` · `prerm` · `postrm` — os scripts que **rodam como root**.

`data.tar.xz`
A árvore de arquivos **como será instalada**:
`./usr/bin/nmap`
`./usr/share/nmap/…`
`./usr/share/man/…`
Nada além de uma cópia dos caminhos de destino.

Instalar é extrair o `data.tar` sobre a raiz e rodar os scripts do `control.tar` nos momentos certos — `preinst` → desempacota → `postinst` na instalação; `prerm` → apaga → `postrm` na remoção. Todos **como root, sem confirmação e sem sandbox**: é assim que um pacote cria usuário, gera chave e registra serviço. Auditar qualquer `.deb` antes é uma sequência de três comandos: `dpkg-deb -I` (metadados) · `dpkg-deb -c` (arquivos) · `dpkg-deb -e` (extrai os scripts, para ler o `postinst` antes de executá-lo).

A COLUNA DO `DPKG -L` : O QUE SE QUER × O QUE É

`ii` `nmap` `7.95+dfsg-3` `amd64` `The Network Mapper`

1º CARACTERE — O QUE SE *QUER*

- `i` instalado
- `r` a remover
- `p` a purgar
- `h` congelado (*hold*)
- `u` desconhecido

2º CARACTERE — O QUE *É*

- `i` instalado e configurado
- `n` não instalado
- `F` configuração falhou
- `c` só a configuração
- `U` desempacotado, sem configurar

`ii` — tudo certo: instalado e configurado. É o que se espera de quase toda linha.

`rc` — removido, configuração ainda em `/etc`. Inofensivo; `apt purge` limpa.

`iU` / `iF` — **quebrado**: desempacotado mas não configurado. **`Bloqueia o APT inteiro.`** Conserto: `sudo dpkg --configure -a` e, se faltar dependência, `sudo apt --fix-broken install`. O `dpkg --audit` lista tudo que está pendente de uma vez.

RELAÇÕES E VERSÕES

- ✓ Além de `Depends`: `Pre-Depends` exige ordem estrita (antes até de desempacotar); `Breaks` / `Replaces` renomeiam pacote e movem arquivos sem conflito; `Provides` cria nome virtual (`mail-transport-agent`).
- ✓ Versão é `época:upstream-revisão`. A **época** conserta numeração que regrediu e nunca diminui; a **revisão** é do empacotador.
- ✓ **O til ordena antes de tudo**, inclusive do vazio: `1.0~rc1` < `1.0`, mas `1.0rc1` > `1.0`.
- ✓ Na dúvida, quem decide é `dpkg --compare-versions '1.10' gt '1.9'`.

INSPEÇÃO — E UM PACOTE SEU

- ✓ `dpkg -L pkg` lista o que ele instalou; `dpkg -S /usr/bin/nmap` responde **de qual pacote veio um arquivo**.
- ✓ `dpkg -V` compara os hashes com o disco: saída vazia = nada alterado desde a instalação.
- ✓ Construir um `.deb` são **dois arquivos e um comando**: a árvore de destino + `DEBIAN/control`, e então `dpkg-deb --build dir pacote.deb`. As linhas extras da `Description` começam **com um espaço** — sem ele o build recusa.
- ✓ Para aplicar patch, recompile o oficial: `apt source` + `apt build-dep` + `dpkg-buildpackage -us -uc -b`, e ponha um `hold` ou revisão `-3.1local1`.

As opções `--force-*` do dpkg quebram o sistema em silêncio. `dpkg -i --force-depends`, `--force-overwrite` e `--force-all` não resolvem erro de dependência: instalam um pacote que **não pode funcionar** e deixam o banco de dados afirmando que está tudo bem. O estrago aparece depois, num contexto sem relação — símbolo indefinido, `apt` que se recusa a operar. Diante de erro de dependência a resposta é sempre `sudo apt --fix-broken install`. Existem para recuperação por quem sabe exatamente o que está fazendo, e só.

RESUMO RÁPIDO

- ✓ O `dpkg` **verifica** dependências mas não as satisfaz — quem busca na rede é o APT.
- ✓ **Prefira sempre** `apt install ./pacote.deb` (com o `./`) a `dpkg -i`: mesma instalação, com as dependências resolvidas antes.
- ✓ Cada pacote instalado tem seus arquivos de controle em `/var/lib/dpkg/info/` (`.list`, `.md5sums`, scripts).
- ✓ `dpkg -l | grep -v '^ii'` mostra tudo que não está normal — o primeiro comando de um diagnóstico.
- ✓ No RHEL: `rpm -qa/-ql/-qf/-V`; o `.rpm` é `cpio` (precisa de `rpm2cpio`) e não tem o estado residual.

NO KALI

Todo `.deb` de terceiro merece auditoria antes de instalar: baixe, `dpkg-deb -e` para extrair os scripts, `dpkg-deb -c` para ver os arquivos — se o `postinst` tem um `curl | bash` para um domínio qualquer, você descobre antes e não depois. **`dpkg -S` e `dpkg -V` são ferramentas de investigação**, não só de administração: "nenhum pacote é dono deste arquivo" para algo em `/usr/bin`, ou um binário do sistema com hash diferente, é achado relevante em verificação de integridade — com a ressalva de que um invasor com root pode ter reescrito os próprios hashes, então é indício, não prova. O sufixo `-0kaliN` identifica o que foi empacotado pelo Kali em vez de vir do Debian. E resolva um `iU` **na hora**: ele bloqueia o APT inteiro, e descobrir isso no meio de um teste custa tempo.